

SpecuNode

Speculative execution for agent graphs, with a store buffer: what happens when an AI agent runs ahead of its own model, and how to make sure nothing it guessed reaches the world

Design and benchmark protocol - version 0, written before the build

Poojan Patel · 15 September 2026

github.com/Poojan6216/specunode · `pip install specunode` (planned; name reserved on PyPI 15 Sep 2026)

READ THIS FIRST - WHAT THIS DOCUMENT IS AND IS NOT

This is the technical report **as it will be published**, written before a line of the runtime exists. It fixes the problem, the design, the invariants, the benchmark protocol and the attacks in advance, so that the numbers cannot be shaped by what turns out to be convenient.

Every result table in this document is therefore empty by design. Cells are filled only by `bench/make_report_pdf.py` from result files committed to the repository. The only measured figures that appear here are cited from published work and are marked **[cited]**.

The first draft of this project, produced by another assistant, contained a latency reduction, a state-divergence rate and an instruction-adherence gain that nobody had measured. None of them survive into this document. Section 4 explains what else was changed and why.

Contents

1. The short version - for anyone
2. The problem: agents wait for the world one call at a time
3. What already exists, and exactly where each stops
4. What was wrong with the first draft, and what replaced it
5. The idea: a store buffer for agents
6. How the runtime works
7. How to use it - install, the three ways in, MCP setup
8. What we will measure
9. The attacks we will run against our own runtime
10. How it is built and tested
11. What this does not do
12. How to reproduce every number
13. Glossary · References

1. The short version - for anyone

An AI agent works in a loop: the model thinks, asks for a tool (look up a customer, restart a job, send an email), waits for the answer, thinks again. Each wait is one to several seconds. A ten-step task spends most of its half-minute waiting for one thing at a time.

The obvious fix is to guess what the model will ask for next and start it early. Researchers at Microsoft showed in March 2026 that this works: agents repeat themselves enough that the next tool call is often predictable, and starting it early cut task time by about half in their tests **[cited]**. But their system has to stop the moment the predicted call would *change* something - restart a job, charge a card - because if the guess is wrong, the change has already happened and cannot be undone. In a real workflow, a write comes along every few steps. That is a low ceiling.

THE CENTRAL IDEA

Do what a CPU does. A processor has been running ahead of itself since the 1960s: it guesses which way a branch goes, runs the next instructions early, keeps any *writes* in a holding area called a store buffer, and only lets them out once the guess is confirmed. If the guess was wrong, the buffer is thrown away and nothing outside the processor ever saw it.

SpecuNode applies that exactly to agents. Reads run early. Writes go into a store buffer tied to the guess that produced them. When the model's real decision arrives, it is compared to the guess - exactly, not approximately. Match: the buffer drains to the world. Mismatch: the buffer is discarded, unsent. The model's real decision is the only thing that can ever release a write.

One thing it deliberately does not do: a held write returns a placeholder instead of a real result, and a placeholder is never shown to the model. The model's next turn after a held write waits for the real result, because otherwise the model would be deciding on a different premise than it would have in an ordinary run. So running ahead past a write hides the time spent on *tools* - the reads and further writes that can proceed in the meantime - not the time spent on the model's next turn. Workloads where every write is immediately read back by the model gain nothing from it, and the benchmark reports how common that shape is.

Two more things make it trustworthy rather than merely fast. Everything the model says and every tool result is written to a journal *before* it is used, so a crashed run resumes where it stopped and a finished run can be replayed exactly - and the replay refuses to continue if anything about the setup has changed, rather than quietly doing something different. And when the runtime cannot be sure a guess is safe to act on - the next call needs a value a held write would have returned, a tool has not said whether it writes, the model's output is prose rather than a decision - it stops guessing and runs that step the ordinary way. It never gambles.

What comes out at the end is an effect ledger: every change that reached the world, the model decision that authorised it, everything that was staged and thrown away, and every point where the runtime gave up on speculation and why. The claim is not that agents get three times faster. The claim is that speculation is now safe past a write, that the runtime can say precisely how much speculation a given workload exposes, and that the list of effects from a speculative run is identical to the list from a plain one.

AND WE WILL PUBLISH WHAT BEATS IT

Section 9 lists nine ways to defeat the runtime, several of which work, and two of which cannot be fixed by any tool at this layer. They will be measured and reported next to the successes. A speed-up that hides its failure modes is a demo, not infrastructure.

2. The problem: agents wait for the world one call at a time

Where the time goes

A multi-step agent's wall clock is a sum: for each step, the model's generation time plus the tool's execution time, one after another. Nothing overlaps. Frameworks parallelise the cases they can see statically - two independent nodes, both branches of a conditional edge - but the common case, a chain where step $i+1$ depends on what the model decided at step i , stays serial.

Why the obvious fix is dangerous

Guessing the next call and running it early is safe only when the call cannot change anything. Three failures follow from running it early when it can:

- **Orphaned effects.** The early write happens; the model decides differently; the write is now in the world with no decision behind it. A charged card, a sent email, a restarted job.
- **Wrong-premise reads.** The early read fetched data that a held write would have changed, or that another actor changed in the meantime, and everything computed from it is stale.
- **Divergent recovery.** The run crashes, is restarted, and the model - which is not deterministic - decides differently the second time. The second run's effects land on top of the first's.

The current state of the art picks one of three answers: do not speculate past writes (loses the speed-up), speculate anyway and clean up with compensating actions (a second effect reaches the world, and for many tools there is no clean undo), or gate every write behind a deterministic check on the serial path (safe, but nothing runs early).

WHY THIS IS A SYSTEMS PROBLEM, NOT A PROMPTING PROBLEM

None of the three failures is caused by the model being wrong. They are caused by the runtime letting an effect out before it had the authority to. The fix is an execution model with a holding stage for effects, exact resolution, and a journal - the same fix processors and databases arrived at decades ago.

3. What already exists, and exactly where each stops

This section was checked against the primary sources on 15 September 2026. The first draft of this project claimed that no one had solved replay divergence and that transactional gating for agents did not exist. Both claims were false; the corrected picture is below, and every project in it is credited in the README.

Work	What it does	Where it stops
PASTE - Act While Thinking (Sui et al., Microsoft Research, arXiv 2603.18897, Mar 2026)	Mines tool-call patterns from agent traces; speculatively pre-executes predicted calls as a tool-serving proxy. Reports 48.5% lower average task time and 1.8x tool throughput [cited].	Side effects handled by <i>policy exclusion</i> : a tool with effects is simply not speculated. No way to hold a write. No journal, no replay. Its follow-ups B-PASTE and Speculate-with-Memory refine the predictor, not the write path.
Claude Code streaming tool executor (described in arXiv 2604.14228)	Starts each tool the instant its block is parsed from the model stream, before the response ends; reads concurrent, writes serial.	Early-issue, not prediction: only runs what the model has already emitted. Writes serialised. This is SpecuNode's Tier-0 drafter, credited as such.
langchain-nvidia-langgraph h (2026)	Compile-time parallelisation; 'speculative execution' runs both arms of a conditional edge and discards the loser.	Its own docs: speculative mode does not support checkpointers, streaming, interrupts or human-in-the-loop. Both arms' tools run for real - the exact failure of Demo 1.
ToolAhead (MCP server)	Prefetches read results for coding agents and replays them on request.	Reads only; writes execute normally and reset prediction.
SagaLLM (Chang & Geng, VLDB 2025)	Saga pattern for multi-agent planning: persistent memory, compensating transactions, validation agents.	Compensation is a second effect reaching the world; an LLM drives the coordinator, which this project forbids in the control path. No speculation.
ATP / Mnemosyne (arXiv 2607.00269, Jul 2026)	A generated action holds no authority until a deterministic gate admits it against an effective-state witness; correctness proven independent of the proposer.	Admission is on the serial path. No notion of an action that executed early and awaits admission. SpecuNode's retirement stage is an ATP-style gate with a store buffer in front of it.
SCOPEGATE (arXiv 2606.28679)	Shows LangChain, LlamaIndex and the Stripe toolkit do not re-authorise each call's argument values; proposes a deterministic per-call gate.	Authorization only. Its finding is why effect classes here are declared out of band and never inferred from the model.
Durable execution - Temporal, DBOS, Restate, Inngest, LangGraph checkpointers	Journal every model output and tool result; replay reuses recorded values; idempotency keys from (workflow, step).	A step is a step: nothing runs before its predecessor completes; no store buffer; no branch to squash. SpecuNode's journal follows the same discipline and is designed so a run can later be hosted inside one of these.

SPECUNODE'S ACTUAL CONTRIBUTION, IN THREE SENTENCES

It is the first runtime that lets an agent graph speculate *past* a mutating tool call, by holding the call's effect in a branch-scoped store buffer that drains only when the target model's real decision confirms the branch, and is discarded - never dispatched - when it does not.

It journals every model output and tool result so that the effect ledger of a speculative run is provably identical to that of the sequential run, and both replay.

It measures, per workload, how much speculation the dependency structure actually exposes past writes, publishing the cases where the answer is 'little' with the same prominence as the cases where it is 'a lot'.

4. What was wrong with the first draft, and what replaced it

The proposal this project started from had the right instinct - speculation plus a transactional outbox - and four problems that would have made it either unbuildable or dishonest. They are recorded here because the corrections *are* the design.

First draft	Problem	What replaced it
'ACID-like' guarantees; 'zero side-effect leakage'; '100% transactional safety'	External APIs cannot participate in an atomic commit. The runtime can guarantee that nothing leaves an unconfirmed branch; it cannot guarantee the world applies a confirmed effect exactly once.	Precise vocabulary: at-least-once dispatch with idempotent dedupe; 'nothing reaches the world from an unretired branch' as a tested invariant; a vocabulary check that fails the build on 'ACID', 'exactly-once', 'guaranteed'.
CRDT state layer merging concurrent speculative branches	Speculative branches are mutually exclusive alternatives. At most one is right. Merging them merges a decision the model made with one it did not make.	Branch isolation: copy-on-write fork per branch; the retiring branch's delta applies, all others are dropped. No merge, ever.
Trajectory pruner + LLM 'distiller' that scrubs rejected turns from the prompt	Unnecessary if each branch forks its own context - rejected turns never enter the canonical history. An LLM summariser in the runtime path breaks determinism and replay.	Per-branch context fork; the confirmed branch's messages become canonical; squashed messages are discarded whole. No model in the control path.
Verification 'y approximately equals y-hat'; acceptance rate 'alpha >= 0.65'	A branch has already acted on y-hat. Any argument difference means every downstream call ran on the wrong premise. 'Approximately' is a way for wrong effects to retire. The 0.65 was invented.	Exact canonical equality on structured decisions; free-text nodes are barriers. Alpha is measured per workload and per drafter tier; the break-even alpha is a reported result.
45.2 s to 14.8 s; 28% divergence; 62% to 94% adherence	Never measured by anyone.	Empty tables in this document; a traceability check that fails CI if a README number appears in no result file.
'Nobody has solved replay divergence'	Durable-execution platforms solved it years ago.	Credited; the journal adopts their record-once-reuse-on-replay discipline and the docs say SpecuNode is not a replacement for them.

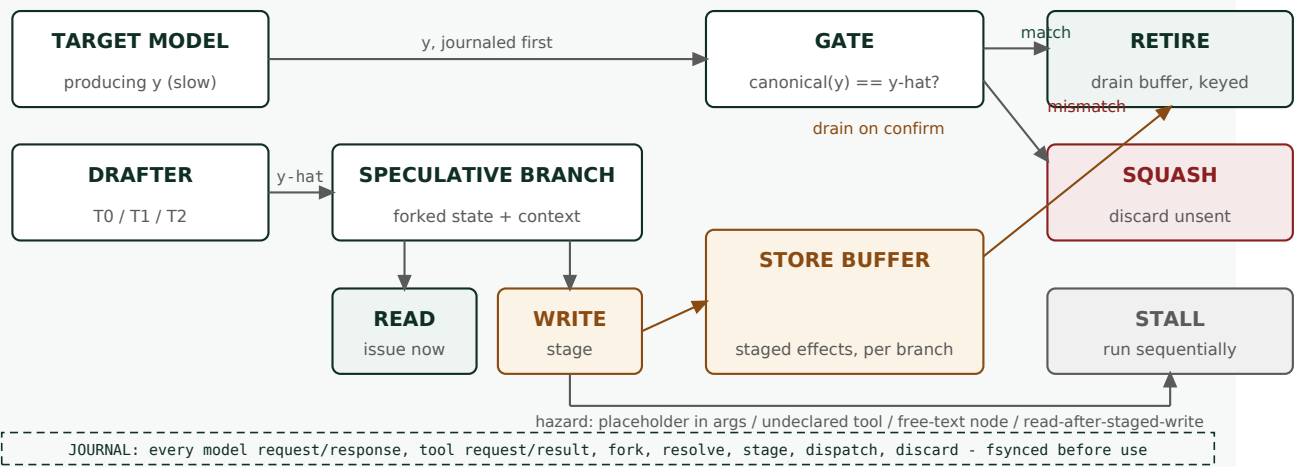
5. The idea: a store buffer for agents

The design is a literal translation of out-of-order execution. The table below is the dictionary; the rest of the report uses the right-hand column.

In a processor	In SpecuNode
Instruction stream	The agent graph: at each node the model emits a decision - a tool call, a route label, or a structured object.
Branch predictor	A drafter : Tier 0 reads decisions off the model's partial stream (always right, but late); Tier 1 predicts from patterns in past runs; Tier 2 asks a small draft model.
Speculative execution	A branch : a forked copy of state and context that runs the next steps on the predicted decision.
Load	A READ tool call - executes immediately, result journaled.
Store buffer	The store buffer : WRITE, COMPENSABLE and IRREVERSIBLE calls are staged, not executed; the branch gets a placeholder handle in place of the return value.
Branch resolution	The gate : the model's real decision, journaled first, compared to the prediction by exact canonical equality.
Retire	Retire : drain the buffer to the world in stage order with deterministic idempotency keys; apply the branch's state delta; its context becomes canonical.
Squash	Squash : cancel the branch task, discard the buffer unsent, drop the context.
Pipeline hazard / stall	Hazard : a call whose arguments contain a placeholder, an undeclared tool, a read of a key a staged write touches with no forwarding, a free-text node, a budget limit. The branch stalls and the runtime continues sequentially from the last confirmed decision.
Store-to-load forwarding	Forwarding : a READ in the same branch of a key a staged WRITE touches can be served from the staged write, if the tool declares its keys and a projection. Otherwise it is a hazard.
Reorder buffer / commit log	The journal and the effect ledger .

One decision point, executed out of order

The model is still producing decision y for step i. The runtime already has a guess y-hat and runs ahead on it.



One decision point. Reads issue on the guess; writes wait; the model's real decision, journaled first, decides whether the buffer drains or is discarded. Hazards stall to sequential.

THE THREE COMMITMENTS THAT FOLLOW FROM THIS

Nothing reaches the world from an unretired branch. A write is dispatched only by retirement, only for a confirmed branch, only after the confirming journal entry is durable. A property test over hundreds of random branch trees asserts it on every build.

Resolution is exact. No similarity, no fuzzy argument matching, no model asked whether two calls are 'the same'. Free text cannot be confirmed by equality, so free-text nodes are barriers.

Hazards stall; they never guess. When the runtime cannot prove a speculative step is safe, that step runs the ordinary way and the ledger says so.

The model is never asked a different question. A speculative branch may call the model only with a prompt the sequential run could have sent: no placeholder in it, tool results in program order, nothing from a discarded sibling. At retirement the prompt is rebuilt from real results and must match byte for byte; a difference is a fault that squashes the branch.

6. How the runtime works

Step one: journal before use

Every request to the target model and every response, every tool request and every result, every fork, resolution, staging, dispatch and discard is appended to a hash-chained journal and fsynced before the runtime acts on it. Replay never calls a model: a replay model serves recorded responses in order. If the request the current code would send at step k differs from the recorded one - a changed prompt, a changed tool list - replay raises a divergence with the step and the diff. It does not re-ask and carry on.

Step two: classify, then either issue or stage

Each tool carries exactly one declared effect class. The developer declares it in code, or the MCP proxy reads it from the server's tool annotations, and a per-tool override table in the config wins over both. A tool with no declaration is a WRITE. No heuristic on a tool's name or description and no model ever assigns a class; this is the runtime's trust boundary, and Section 9 measures what happens when it is lied to.

Class	On the speculative path	At retirement
READ	Executes immediately. Result and, if the tool provides one, a version witness are recorded in the branch's read-set.	Witnessed reads are re-validated; a changed witness squashes the branch (default) or stalls it. Unwitnessed reads are reported, never counted fresh.
WRITE	Staged in the store buffer. The branch receives a placeholder handle. If a later call's arguments contain the handle, or the branch's next model call would show it: hazard, stall.	Dispatched in stage order with an idempotency key derived from run, branch lineage, step, tool and canonical arguments. At-least-once with dedupe on the key.
COMPENSABLE	As WRITE, plus the compensator is recorded.	As WRITE. If a later decision needs it undone, the compensator is dispatched as a new effect and the ledger shows both.

Class	On the speculative path	At retirement
IRREVERSIBLE	A barrier by default: the branch stalls before it. A config flag permits staging; the docs explain the trade.	If staged and confirmed, dispatched like WRITE, with the ledger row marked irreversible.

A READ THAT ENQUEUES IS A WRITE

A tool whose synchronous response is harmless - 'status: queued, job 9921' - but whose upstream then runs a background job that writes and notifies is a write in disguise. Declared READ, it runs early; if the branch is thrown away, the job runs anyway. The runtime cannot see this from inside the process. Any tool that enqueues, schedules or triggers work asynchronously must be declared WRITE, whatever its HTTP verb, and the documentation says so on its first page.

Step three: resolve, then retire or squash

When the journaled model response for the step arrives, every open branch at that step is resolved at once. Exactly one can be confirmed; the rest are squashed, their tasks cancelled, their buffers discarded with a journal entry recording the count. A confirmed branch first re-validates its witnessed reads; only then does it drain. The ordering matters: **'a read went stale' is decided before 'the decision matched'**, so a branch whose premise changed under it cannot retire on a lucky match.

Step four: survive a crash, then hand back a ledger

Because every effect is journaled before dispatch and every dispatch is keyed on run, branch lineage, node, a journaled step counter and canonical arguments, a run killed at any point resumes from the journal: already-dispatched effects are not re-sent, an in-flight drain continues. The result of a run is an Ed25519-signed effect ledger: one row per effect that reached the world with the journal offset of the model decision that authorised it, plus the counts of squashed branches, discarded effects, stalls by reason, reads validated, wasted tokens, and the drafter's acceptance rate over the run.

What an effect ledger can claim

DISPATCHED

proves: the target model made this decision, and it was sent once per ledger

but: the world may still duplicate it

DISCARDED

proves: a squashed or stalled branch staged it and it was never sent

but: its READs already reached upstream

STALLED

proves: a hazard was seen and the runtime fell back to sequential

but: the speedup for that step is zero

UNWITNESSED

proves: a read was used but could not be re-validated

but: never reported as fresh

A ledger with zero DISCARDED and zero STALLED rows on a speculative run is suspicious, not impressive. The report prints both.

THE RULE THAT KEEPS SPECULATION HONEST

For any journaled run, the effect ledger with speculation on must equal the effect ledger with speculation off, after stripping timestamps and branch ids. This is the equivalence test. It runs on every workload, every drafter tier, every CI job, and a failure is a build failure, not a flaky test. Speculation is allowed to change *when* things happen. It is not allowed to change *what* happens.

7. How to use it - install, the three ways in, MCP setup

Published as `specunode` on the Python Package Index (name reserved; unpublished at the time of writing). Python 3.11 or newer - the floor was lowered from a previous project's 3.12 after its first two real installs failed on 3.11 interpreters with a misleading pip message.

```
uv tool install specunode          # the command: specunode
uv tool install 'specunode[mcp]'   # plus the MCP proxy
uv tool install 'specunode[local-draft]' # optional Tier-2 local draft model

cd /path/to/your-agent
specunode init                     # writes specunode.yaml and .specunode/
```

Way one - wrap a LangGraph graph, unchanged

```
from specunode.integrations.langgraph import wrap
from specunode import tool

@tool(effect="read", witness=True)
async def get_pipeline_status(job_id: str): ...

@tool(effect="write", idempotent=True, forward_keys=lambda a: {f"job:{a['job_id']}"})
async def restart_job(job_id: str): ...

@tool(effect="irreversible")
async def post_summary(channel: str, text: str): ...

app = wrap(graph.compile())          # your graph file does not change
result = await app.ainvoke({"task": "etl-7 is stuck"})
print(result.ledger)
```

The graph definition is untouched. Nodes' tool calls are routed through the runtime's tool runner and their model calls through the journaled client. Interrupts on the canonical path work as in vanilla LangGraph; an interrupt inside a speculative branch is a hazard and stalls.

Way two - a plain Python loop

```
from specunode import run_loop, node

@node
async def decide(ctx): return await ctx.model.decide(ctx.messages) # returns a Decision

result = await run_loop(decide, tools=[...], config="specunode.yaml")
```

Way three - the MCP proxy, for agents you cannot change

Step	What to do
1. Install the extra	<code>uv tool install --force 'specunode[mcp]'</code>
2. Point the proxy at your server	<code>specunode mcp-proxy --upstream "npx my-mcp-server" --config /full/path/specunode.yaml</code> - the config path must be absolute.
3. Point your client at the proxy	Replace the server command in the client's MCP config with the proxy command. <code>tools/list</code> passes through with effect-class annotations merged from your override table.
4. Tell it what the model decided	The LangGraph and plain integrations do this automatically. A foreign client sends <code>notifications/specunode/decision</code> , or a human runs <code>specunode retire <run> --step N --decision '<json>'</code> . Until then, staged writes stay staged and the <code>specunode.status</code> tool says so.

WHAT THE PROXY WILL NOT DO

It will not guess a tool's effect class. MCP servers that ship no annotations get every tool classified `WRITE`, which means no speculation past any of them until you fill in the override table. That is deliberate: the per-call authorisation work (SCOPEGATE) showed what happens when a runtime infers authority from a tool's description.

It will not discard a branch on an assistant's say-so without a human confirmation on protocol versions that support elicitation; on versions that do not, it refuses and prints the terminal command.

Reading a ledger

```
EFFECT LEDGER run 01K5... speculation=on drafters=T0,T1 target=<model>
# effect key branch decided-by(step) status
1 restart_job {id:"etl-7"} b2f9... br-03 step 4 (retired) DISPATCHED ack=1
2 post_summary {ch:"#ops"} 1lac... br-03 step 4 (retired) DISPATCHED ack=1
squashed branches: 2 staged effects discarded: 3 stalls: 1 (free-text node "draft_reply")
reads validated at retirement: 4/4 fresh unwitnessed reads: 0 speculative reads upstream:
-
context divergences: 0 wasted tokens: - alpha(20): -
```

Which reads as: two effects reached the world, both authorised by the model's decision at step 4. Two guesses were wrong and their three staged effects were thrown away unsent. Once, the runtime met a prose-emitting node and fell back to running that step the ordinary way. Every read used by the winning branch was re-checked at retirement and had not changed. The dashes are where measured values go.

When something goes wrong

What you see	What it means
tool 'x' has no declared effect class; treating as WRITE	Declare it, or accept that nothing speculates past it.
STALLED: RETURN_VALUE_DEPENDENCY at step 6	A later call needed the value a staged write would return. Correct behaviour. If the tool can accept a handle, declare it; otherwise this step is inherently serial.
ReplayDivergence(step=1, diff=...)	The prompt, tool list or model differs from the journaled run. Replay refuses to continue on a different premise.
speculation disabled by policy (alpha 0.20 < break-even 0.45)	The drafter is wrong too often on this workload to pay for itself. The run continues sequentially. Both numbers come from the run.
DEAD_LETTER: dispatch of effect 3 failed after 5 attempts	The run finished ok=False. Nothing was silently skipped. Resume after the upstream recovers.

8. What we will measure

Three questions, in this order, because the first one bounds the other two: **how much speculation is there to have**, **what does it cost when wrong**, and only then **how much faster is it**. All cells are '-' until produced by committed runs; every row will carry a bootstrap 95% confidence interval over traces or tasks.

8.1 Opportunity: how much speculation does a workload expose? (offline, no model calls)

For each trace in the corpus - a seeded 2,000-trajectory sample of nebius/SWE-rebench-openhands-trajectories (Hugging Face, CC-BY-4.0, 67,074 OpenHands trajectories with per-step tool names and JSON arguments; verified available on 15 September 2026), falling back to traces generated by the three sample apps if it disappears - the analysis computes, with effect classes taken from a committed hand-written table (unlabelled tools are WRITE, as in the runtime):

Measure	Read-only speculation (PASTE policy)	SpecuNode (writes stage)
Next call predictable by Tier-1, top-1 / top-3 (leave-one-trace-out)	-	-
Predicted calls that are READ / WRITE / other	-	-
Median speculable run length before a barrier (steps)	-	-
Steps blocked, by hazard: return-value dependency / undeclared / free text / read-after-write	-	-
Steps behind free-text barriers	-	-
Steps that are a model call immediately consuming a write's result (no past-write gain possible)	-	-

The comparison of the third row across the two columns is the project's first headline number: how far past a write the runtime can run when writes are staged rather than forbidden. If the answer is 'barely further' on a workload, that is published as such.

8.2 Cost of being wrong

Measure	Support agent	Ops agent	Research agent
Acceptance rate alpha, Tier 0 / Tier 1 / Tier 2	-	-	-
Wasted tokens per run (draft + target on squashed branches)	-	-	-
Speculative READs that reached upstream from squashed branches	-	-	-
Stale witnessed reads at retirement / unwitnessed reads	-	-	-
Break-even alpha (where speculative wall clock = sequential)	-	-	-

8.3 Latency (online, real target model, spend-capped)

Three sample apps, 30 seeded tasks each, four runtimes sharing one journal so they replay the same model outputs. The spend cap is recorded in the report; if it is hit, the reduced sample size and wider interval are reported rather than the cap raised.

Runtime	Wall clock, median [CI]	vs sequential	Effects dispatched	Effects from squashed branches
Sequential (speculation off)	-	1.00	-	0 (by construction)
Read-only speculation (PASTE policy, credited)	-	-	-	0 (by construction)
Naive parallel (writes execute, branch discarded)	-	-	-	- (expected > 0)
SpecuNode	-	-	-	0 (asserted by the leak test)

8.4 Overhead

The journaling and classification overhead of a sequential SpecuNode run over the same graph on vanilla LangGraph with the same replay model, in milliseconds per step and as a fraction of wall clock. Speculation must never make a run slower than sequential by more than this overhead; the policy gate enforces it and the bench confirms it.

WHAT WOULD COUNT AS A NEGATIVE RESULT, STATED IN ADVANCE

If the measured break-even alpha exceeds the measured Tier-1 acceptance rate on every workload, the headline becomes: 'early-issue plus the store buffer is the useful part; pattern drafting did not pay for itself on these workloads'. The workloads will not be tuned until that flips. Decision gate D3 in the build spec says so.

9. The attacks we will run against our own runtime

Ten strategies. Each is one file that returns a measured rate; a strategy that fails to run is reported as an error row, never dropped. Results appear in the README under 'What beats it', placed before the wins.

Attack	In plain terms	Expected outcome
9.1 Misdeclared tool	A tool declared READ that actually writes.	Effects from squashed branches reach the world. Cannot be fixed by the runtime; it is the trust boundary.
9.2 Hidden side effects in a read	A READ that bills, rate-limits or audits upstream.	Squashed branches' reads reach upstream. Counted in every ledger, never hidden.
9.3 Stale reads under contention	Another actor mutates rows between the speculative read and confirmation, at 1/10/100 per second.	Witnessed stale reads squash the branch. The honest number is the fraction that were <i>unwitnessed</i> and therefore undetectable.
9.4 Non-idempotent tool, duplicate delivery	The network delivers a dispatch twice.	Dedupe protects the dispatcher's own retries, not the network beyond it. Measured with and without the dedupe table.
9.5 Return-value laundering	A placeholder handle copied into a string, nested, or base64-encoded before a later call.	Substring scan catches plain and nested; encoded evades. Miss rate reported; encoded documented as unfixable here.
9.6 Prompt-injected tool call	A read result tells the model to send an email.	If only the drafter predicts it: squash, zero dispatches. If the target model emits it: dispatched. SpecuNode is not authorization; the README points to per-call policy as the missing layer.
9.7 Drafter poisoning	Train the pattern index on traces with an adversarial chain ending in a write.	Wasted tokens and stalls rise; leaks stay at zero.
9.8 Replay under drift	Replay after changing the prompt by one token; after changing the tool list.	Divergence at step 1 in both cases.
9.9 Staging an IRREVERSIBLE effect	Config flag on.	Faster, and the ledger shows an irreversible effect retired on a real decision. The default stays off and the docs say why.
9.10 Asynchronous side effect behind a READ	A READ whose upstream enqueues a job that writes and notifies after replying 'queued'.	The branch is squashed; the job runs anyway. Leaked effects per squashed branch expected non-zero. Unfixable at this layer; such tools must be declared WRITE.

THE THREE THAT CANNOT BE FIXED

A read that enqueues. The synchronous reply is harmless; the effect happens later, elsewhere. Declared READ, it runs early and the effect leaks when the branch is squashed. The declaration is the only defence.

A lie about effects. If a tool says it only reads and it writes, speculation runs it early and the write happens. No runtime can see through a declaration from inside the process. The only defence is the declaration, and the docs say so on the first page.

Reads without a witness. A read that carries no version cannot be re-checked at retirement. The runtime reports it as unwitnessed and refuses to call it fresh; it cannot make it fresh. Tools that want their reads validated must return a version, and the sample apps show how.

10. How it is built and tested

Part	Choice	Why
Language	Python 3.11+, asyncio TaskGroup	Where agent frameworks live; task cancellation is the squash primitive.
Journal	SQLite WAL, synchronous=FULL, hash-chained; Postgres optional	Durable before use with no extra service; one DDL for both.
Canonical form	Sorted-key JSON, NFC, repr floats, blake2b	Exact resolution and stable idempotency keys need one byte form per value.
State	Copy-on-write fork per branch, RFC 6902 delta at retirement	Isolation, not merge.
Drafters	T0 stream early-issue (Claude Code's pattern); T1 pattern index (PASTE's); T2 draft model	Three tiers with three different alphas and costs, measured separately.
Integrations	LangGraph wrapper, plain-Python decorators, MCP stdio proxy	Same core, three doors.
Fake world	In-memory tables with a branch-tagged mutation log and fault injection	The leak test needs a world that records who touched it.
Ledger	Ed25519-signed, hash-chained to the journal	Tamper-evident; verifiable without trusting the runtime.

The tests that can never be skipped

- **The leak test.** Hundreds of randomly generated branch trees with random resolutions and random faults; after each, every mutation in the world must belong to a retired branch.
- **The equivalence test.** Every workload, every tier: ledger with speculation on equals ledger with speculation off.
- **The context-equivalence test.** Each sample app run sequentially and speculatively against a model stub that records every prompt it receives; the prompts at every canonical step must be byte-identical, and every prompt a speculative branch sent must equal the canonical one at that step. Replay-based equivalence cannot catch a placeholder reaching a prompt; this test can.

- **The kill test.** The process is killed at 15 random points; resume must reproduce the uninterrupted ledger byte for byte.
- **The honesty tests.** A vocabulary check fails the build on 'ACID', 'exactly-once', 'guaranteed', 'zero-latency', 'eliminates', '100% safe' or 'context rot' without a qualifier; a traceability check fails it on any README number that appears in no result file; a grep fails it on any prompt inside the control-path packages.

What it will cost to measure

The offline analysis and all correctness tests run on a laptop CPU with no model calls. The online latency bench calls a real target model under a spend cap recorded in the results; the report states the spend and the sample size actually achieved.

11. What this does not do

Stated plainly, because a runtime that oversells its safety is worse than none at all.

- **It is not authorization.** A call the target model actually emits is dispatched. Per-call policy on argument values is a separate layer.
- **It cannot see through a tool's declaration.** A READ that writes defeats it entirely.
- **Speculative reads reach upstream** even when the branch is thrown away. Reads with billing or audit side effects are a speculative cost, counted in every ledger.
- **It cannot validate a read that carries no witness.** It reports the read as unwitnessed, never as fresh.
- **Idempotency dedupe covers its own retries,** not duplication beyond the dispatcher. Tools that must be exactly-once need their own keys.
- **Past-write speculation hides tool latency, not model latency.** The model call after a held write waits for the real result. Chains shaped model, write, model-reads-the-result gain nothing, and the bench reports that fraction.
- **A read that enqueues is a write in disguise.** The runtime cannot detect it; the declaration must say WRITE.
- **Free-text nodes are barriers.** Workloads that hand prose from node to node get little or no speed-up, and the bench says how much.
- **It is not a durable-execution platform.** The journal is local. Hosting a run inside Temporal, DBOS or Restate is documented as a pattern, not shipped.
- **It is not a context manager.** It never prunes, summarises or distils a prompt. Rejected branches simply never enter the canonical history.
- **It is a research prototype:** one model-provider adapter, three integrations, three sample apps, one scope at a time.

WHAT IT IS

A runtime that lets an agent run ahead of its own model, holds every write until the model has actually decided it, throws away what it guessed wrong without the world ever seeing it, replays exactly, and hands back a ledger that says which decision authorised each effect and where it had to stop guessing. That is a narrower claim than the first draft made, and it has the advantage of being checkable.

12. How to reproduce every number

Every figure in the published version of this report will be produced by one of these commands and written to a result file committed to the repository. The report, the README and RESULTS.md are generated from those files; nothing is typed in by hand.

To reproduce	Command
The opportunity analysis (8.1)	<code>uv run python bench/offline/run_opportunity.py --all</code>
The cost-of-being-wrong and latency tables (8.2, 8.3)	<code>uv run python bench/online/run_latency.py --all</code>
The overhead figure (8.4)	<code>uv run python bench/online/run_latency.py --overhead</code>
The chaos matrix	<code>uv run python bench/chaos/run_chaos.py --all</code>
The attacks (9)	<code>uv run python bench/adversarial/run_attacks.py --all</code>
The three demos	<code>uv run python bench/demo.py --demo leak past-write replay</code>
RESULTS.md and the plots	<code>uv run python bench/report.py && uv run python bench/plots/make_plots.py</code>
The full test suite	<code>uv run pytest</code>
This report	<code>uv run python bench/make_report_pdf.py</code>

Hardware, Python version, compute time and bench spend are recorded by the bench scripts and printed here in the published version.

13. Glossary

Decision

What the model emits at a node: a tool call with arguments, a route label, or a structured object. Free text is a decision the runtime cannot speculate on.

Branch

A speculative continuation of the graph from a predicted decision, with its own copy of state and context. Ends retired, squashed or stalled.

Store buffer

Where staged writes wait. Named after the processor structure that holds speculative stores until the branch is confirmed.

Retire / squash / stall

Confirm a branch and drain its buffer; discard a branch and its buffer unsent; give up speculating at a hazard and run the step sequentially.

Hazard

A condition under which the runtime cannot prove a speculative step is safe: a placeholder in the arguments, an undeclared tool, a read of a key a staged write touches, a free-text node, a budget limit.

Witness

A version or ETag returned with a read, so the read can be re-validated at retirement.

Idempotency key

A hash of run, branch lineage, step, tool and canonical arguments; the same logical effect always gets the same key, so retries and resumes never send it twice from this runtime.

Drafter / alpha

The component that guesses the next decision, and the fraction of its guesses the model confirms.

Effect ledger

The signed record of every effect that reached the world, the decision that authorised it, and everything staged and discarded.

Equivalence

The property that the ledger with speculation on equals the ledger with speculation off for the same journaled model outputs.

Context identity

The property that every prompt a speculative branch sends to the model is byte-identical to the prompt the sequential run would send at that step. Its live test is the context-equivalence test.

14. References

- Sui, Zhao, Ma, He, Wang, Li, Yang. *Act While Thinking: Accelerating LLM Agents via Pattern-Aware Speculative Tool Execution (PASTE)*. arXiv 2603.18897, March 2026. The pattern-mining drafter re-implements its idea and credits it.
- B-PASTE, arXiv 2604.16469; *Speculate with Memory*, arXiv 2607.12236 - follow-ups on the predictor.
- *Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems*, arXiv 2604.14228 - the streaming tool executor that is the Tier-0 drafter.
- LangChain and NVIDIA, *langchain-nvidia-langgraph*, 2026 - conditional-edge speculation and its documented limits.
- ToolAhead, MCP server - read prefetching for coding agents.
- Chang and Geng. *SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning*. VLDB 2025, arXiv 2503.11951.
- *Mnemosyne: Agentic Transaction Processing for Validating and Repairing AI-generated Workflows*. arXiv 2607.00269, July 2026 - the admission-gate model behind retirement.
- *Capability Gates Are Not Authorization: Confused-Deputy Failures in LLM Agent Frameworks (SCOPEGATE)*. arXiv 2606.28679.
- Temporal, DBOS, Restate, Inngest documentation on durable execution for agents; LangGraph checkpoints - the journal's record-once, reuse-on-replay discipline.
- Tomasulo, R. M. *An Efficient Algorithm for Exploiting Multiple Arithmetic Units*. IBM Journal, 1967 - the store-buffer-and-retirement model this design copies.